
Python Standard Library

Interview Reference Guide

VCU Health — AI Engineer II Interview Prep

Zarak Shah Ji

March 2026

Section	Priority	Modules
1. itertools	HIGH	chain, groupby, product, combinations, islice
2. functools	HIGH	lru_cache, reduce, partial, wraps
3. dataclasses	HIGH	@dataclass, field, asdict, frozen
4. json	HIGH	dumps/loads, dump/load, custom encoders
5. typing	HIGH	Optional, Union, Callable, Protocol
6. re	MEDIUM	search, match, findall, sub, compile
7. datetime	MEDIUM	timedelta, strptime/strftime, isoformat
8. logging	MEDIUM	getLogger, levels, basicConfig
9. os / pathlib	MEDIUM	Path, glob, environ
10. csv	LOWER	DictReader, DictWriter
11. contextlib	LOWER	@contextmanager, suppress
12. abc	LOWER	ABC, @abstractmethod
Bonus	—	Counter, enum, Protocol, concurrent.futures

HIGH PRIORITY MODULES

1. itertools — Efficient Lazy Iteration

Key insight: itertools returns **lazy iterators** — memory-efficient on large datasets (think millions of EHR records). Nothing is materialized until consumed.

`chain(*iterables)` — Flatten multiple iterables into one stream

```
from itertools import chain

# Combine patient records from multiple hospital departments
icu_patients = ["P001", "P002"]
er_patients = ["P003", "P004"]
all_patients = list(chain(icu_patients, er_patients))
# ["P001", "P002", "P003", "P004"]

# chain.from_iterable — when you have a list of lists
dept_lists = [icu_patients, er_patients]
all_patients = list(chain.from_iterable(dept_lists))
```

`groupby(iterable, key)` — Group consecutive elements

```
from itertools import groupby

visits = [
    {"patient": "P001", "dept": "cardio"},
    {"patient": "P002", "dept": "cardio"},
    {"patient": "P003", "dept": "neuro"},
]
# MUST be sorted by key first!
for dept, group in groupby(visits, key=lambda v: v["dept"]):
    patients = list(group)
    print(f"{dept}: {len(patients)} visits")
```

■ **GOTCHA:** Data **must be pre-sorted** by the groupby key. Unsorted data produces fragmented groups. This is the #1 interview gotcha for groupby.

`product(*iterables)` — Cartesian product

```
from itertools import product

# Test all drug-dosage combinations
drugs = ["metformin", "insulin"]
dosages = ["low", "medium", "high"]
combos = list(product(drugs, dosages))
# 2 x 3 = 6 combinations
```

`combinations(iterable, r)` vs `permutations(iterable, r)`

```
from itertools import combinations, permutations

biomarkers = ["A1C", "glucose", "BMI", "BP"]

# combinations: order doesn't matter, no repeats
pairs = list(combinations(biomarkers, 2)) # 6 pairs (4C2)

# permutations: order matters
ordered = list(permutations(biomarkers, 2)) # 12 results (4 x 3)
```

■ **TIP:** combinations = "which pairs of features to try in a model"
permutations = "ordered sequences matter (e.g., treatment order)"

islice(iterable, start, stop, step) — Slice any iterator

```
from itertools import islice

# Grab first 100 records from a streaming EHR query
first_100 = list(islice(ehr_stream, 100))

# Skip first 10, take next 50
batch = list(islice(ehr_stream, 10, 60))
```

Bonus: accumulate, zip_longest, starmap

```
from itertools import accumulate, zip_longest

# Running totals – e.g., cumulative patient admissions
daily = [12, 8, 15, 10]
cumulative = list(accumulate(daily)) # [12, 20, 35, 45]

# Zip with fill value – align unequal-length lists
names = ["Alice", "Bob"]
scores = [85, 90, 78]
list(zip_longest(names, scores, fillvalue="N/A"))
# [("Alice", 85), ("Bob", 90), ("N/A", 78)]
```

2. functools — Higher-Order Function Utilities

`lru_cache(maxsize=128)` — Memoization decorator

```
from functools import lru_cache

# Cache expensive lookups – e.g., ICD code descriptions
@lru_cache(maxsize=1024)
def get_icd_description(code: str) -> str:
    return icd_database.query(code) # expensive DB call

get_icd_description("E11.9") # Type 2 diabetes – hits DB
get_icd_description("E11.9") # Cached! Instant.

# Check stats and clear
print(get_icd_description.cache_info())
get_icd_description.cache_clear()
```

■ **GOTCHA:** Arguments must be **hashable** (no lists or dicts). Use `maxsize=None` for unbounded cache.

`reduce(function, iterable[, initializer])` — Fold to single value

```
from functools import reduce

# Cumulative multiplicative risk score
risk_factors = [1.2, 0.8, 1.5, 1.1]
cumulative = reduce(lambda a, b: a * b, risk_factors) # 1.584
```

■ **TIP:** In practice, `sum()` or `math.prod()` is clearer. Know `reduce` exists but don't overuse it.

`partial(func, *args, **kwargs)` — Freeze some arguments

```
from functools import partial

def calculate_bmi(weight_kg, height_m):
    return weight_kg / (height_m ** 2)

# Specialized function with fixed height
bmi_for_patient = partial(calculate_bmi, height_m=1.75)
bmi_for_patient(80) # 26.12
```

`wraps(wrapped)` — Preserve metadata in decorators

```
from functools import wraps
import logging

def audit_log(func):
    @wraps(func) # Without this, __name__ becomes "wrapper"
    def wrapper(*args, **kwargs):
        logging.info(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@audit_log
def update_patient_record(patient_id, data):
    ...
# func.__name__ still "update_patient_record", not "wrapper"
```

3. dataclasses — Structured Data with Minimal Boilerplate

Bridges data structures and OOP. Auto-generates `__init__`, `__repr__`, `__eq__`. Interview favorite because it shows clean, maintainable code.

```
from dataclasses import dataclass, field, asdict
from typing import Optional
from datetime import datetime

@dataclass
class PatientEncounter:
    patient_id: str
    encounter_date: datetime
    department: str
    diagnosis_codes: list[str] = field(default_factory=list)
    risk_score: Optional[float] = None
    is_readmission: bool = False

encounter = PatientEncounter(
    patient_id="P001",
    encounter_date=datetime(2025, 3, 10),
    department="cardiology",
    diagnosis_codes=["I50.9", "E11.9"]
)
asdict(encounter) # Convert to dict for JSON serialization
```

Key Features: frozen, order, `__post_init__`

```
# Frozen (immutable) – good for reference data
@dataclass(frozen=True)
class ICD10Code:
    code: str
    description: str

# order=True – enables < > sorting
@dataclass(order=True)
class RankedDiagnosis:
    probability: float # sorted by this first
    code: str

# Post-init processing
@dataclass
class LabResult:
    value: float
    unit: str
    reference_low: float
    reference_high: float
    is_abnormal: bool = field(init=False)

    def __post_init__(self):
        self.is_abnormal = not (
            self.reference_low <= self.value <= self.reference_high
        )
```

■ **GOTCHA:** Never use mutable defaults directly: `items: list = []` raises `TypeError`. Always use `field(default_factory=list)`.

When to use what:

- **dataclass** — Mutable structured data with methods, validation, type hints. Most versatile.
- **NamedTuple** — Immutable, lightweight, tuple-like behavior with named fields.
- **dict** — Keys are dynamic/unknown at design time.

4. json — Serialization

```
import json
from datetime import datetime

# Basics
data = {"patient_id": "P001", "scores": [0.85, 0.92]}
json_str = json.dumps(data) # dict -> string
json_str = json.dumps(data, indent=2) # pretty-printed
parsed = json.loads(json_str) # string -> dict

# File I/O
with open("patient.json", "w") as f:
    json.dump(data, f) # dict -> file
with open("patient.json") as f:
    data = json.load(f) # file -> dict

# Custom encoder for non-serializable types (datetime, etc.)
class HealthcareEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

record = {"timestamp": datetime.now(), "a1c": 6.5}
json.dumps(record, cls=HealthcareEncoder)
```

■ **GOTCHA:** **dumps/loads** = string operations. **dump/load** = file operations. Only basic Python types are serializable — datetime, set, dataclass all need custom encoders.

5. typing — Type Hints

```
from typing import Optional, Union, Any
from collections.abc import Callable, Sequence

# Modern Python 3.10+ uses built-in types
def get_risk_scores(patient_ids: list[str]) -> dict[str, float]:
    ...

# Optional = Union[X, None]
def find_patient(mrn: str) -> Optional[dict]:
    ... # might return None

# Callable for function parameters
def apply_transform(
    data: list[float],
    transform: Callable[[float], float]
) -> list[float]:
    return [transform(x) for x in data]

# Type aliases for readability
PatientID = str
RiskScoreMap = dict[PatientID, float]

# Python 3.10+ pipe syntax
def process(val: int | float | str) -> float: # replaces Union
    ...
def maybe(mrn: str) -> dict | None: # replaces Optional
    ...
```

■ **TIP:** Type hints are **NOT enforced at runtime** — they're for readability, tooling (mypy), and documentation. In 3.10+: `str` | `None` replaces `Optional[str]`.

MEDIUM PRIORITY MODULES

6. re — Regular Expressions

```
import re

# Core functions
re.search(pattern, string) # First match ANYWHERE -> Match or None
re.match(pattern, string) # Match only at BEGINNING -> Match or None
re.findall(pattern, string) # All non-overlapping matches -> list
re.sub(pattern, repl, string) # Replace all matches

# Extract MRN from clinical note
note = "Patient MRN: 12345678 presented with chest pain"
match = re.search(r"MRN:\s*(\d+)", note)
if match:
    mrn = match.group(1) # "12345678"

# Find all ICD-10 codes in a document
text = "Diagnoses: E11.9, I50.9, J44.1"
codes = re.findall(r"[A-Z]\d{2}\.\d{1,2}", text)
# ["E11.9", "I50.9", "J44.1"]

# De-identification: redact phone numbers (HIPAA)
note = "Call patient at 804-555-1234"
redacted = re.sub(r"\d{3}[-.]\d{3}[-.]\d{4}", "[REDACTED]", note)

# Compile for performance in loops
icd_pattern = re.compile(r"[A-Z]\d{2}\.\d{1,2}")
codes = icd_pattern.findall(text)
```

■ **GOTCHA:** **search** scans the whole string. **match** only checks the beginning. Use `re.compile()` when reusing the same pattern in a loop.

7. datetime — Date and Time Operations

```
from datetime import datetime, date, timedelta

# Parse and format
dt = datetime.strptime("2025-03-13", "%Y-%m-%d") # string -> datetime
formatted = dt.strftime("%m/%d/%Y") # datetime -> string "03/13/2025"

# Current time
now = datetime.now()
today = date.today()

# Date arithmetic - readmission windows, length of stay
admission = datetime(2025, 3, 1)
readmission_window = admission + timedelta(days=30)
discharge = datetime(2025, 3, 5)
los = (discharge - admission).days # Length of stay = 4

# ISO format (what FHIR APIs use)
dt.isoformat() # "2025-03-13T00:00:00"
datetime.fromisoformat("2025-03-13T14:30:00")
```

■ **TIP:** Mnemonic: **strptime** = **parse** (string->datetime). **strftime** = **format** (datetime->string). Common codes: %Y (4-digit year), %m (month), %d (day), %H:%M:%S (time).

8. logging — Production-Grade Logging

```
import logging

# Basic config – do once at the top
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger(__name__)

# Log levels: DEBUG < INFO < WARNING < ERROR < CRITICAL
logger.debug("Model input shape: %s", X.shape) # dev only
logger.info("Training started for cohort A")
logger.warning("Missing values in 15% of records")
logger.error("Inference failed for patient %s", pid)
logger.critical("Database connection lost")
```

■ **TIP: Why logging over print?** Levels, timestamps, file/service output, per-module control. In production ML, you need auditable logs — print disappears.

9. os / pathlib — File System Operations

pathlib is the modern way. Know both, prefer pathlib.

```
from pathlib import Path
import os

# pathlib (preferred)
p = Path("/data/ehr/patient_records")
p.exists() # True/False
p.is_file() # True/False
p.mkdir(parents=True, exist_ok=True) # Create dirs recursively
p.stem # filename without extension
p.suffix # ".csv"

# Path joining – use / operator
config_path = Path("/app") / "config" / "model.yaml"

# Glob for file discovery
csv_files = list(p.glob("*.csv")) # current dir
all_csvs = list(p.glob("**/*.csv")) # recursive

# Read/write shortcuts
text = Path("notes.txt").read_text()
Path("output.txt").write_text("results")

# os module – env vars (still useful)
region = os.environ.get("AWS_REGION", "us-east-1")
```

LOWER PRIORITY MODULES

10. csv — CSV File Processing

```
import csv

# DictReader — returns dicts keyed by headers
with open("patients.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["patient_id"], row["diagnosis"])

# DictWriter
with open("output.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["patient_id", "risk"])
    writer.writeheader()
    writer.writerow({"patient_id": "P001", "risk": 0.85})
```

■ **TIP:** In practice, use Pandas for CSVs. But DictReader is lighter for streaming large files without loading everything into memory.

11. contextlib — Custom Context Managers

```
from contextlib import contextmanager, suppress
import time

@contextmanager
def timer(label: str):
    start = time.time()
    yield
    elapsed = time.time() - start
    print(f"{label}: {elapsed:.2f}s")

# Usage
with timer("Model inference"):
    predictions = model.predict(X_test)
# "Model inference: 0.34s"

# Suppress specific exceptions
from pathlib import Path
with suppress(FileNotFoundError):
    Path("missing.txt").unlink()
```

12. abc — Abstract Base Classes

```

from abc import ABC, abstractmethod

class BaseModel(ABC):
    @abstractmethod
    def train(self, X, y): ...

    @abstractmethod
    def predict(self, X): ...

    def evaluate(self, X, y): # Shared, concrete method
        preds = self.predict(X)
        return accuracy_score(y, preds)

class ReadmissionModel(BaseModel):
    def train(self, X, y): # Must implement or TypeError
        ...
    def predict(self, X):
        ...

```

■ **TIP:** Shows you can design extensible ML systems. If VCU builds multiple model types, ABCs enforce a consistent interface.

BONUS: Worth Knowing

`collections.Counter`

```
from collections import Counter
diagnoses = ["E11.9", "I50.9", "E11.9", "J44.1", "E11.9"]
Counter(diagnoses).most_common(2) # [("E11.9", 3), ("I50.9", 1)]
```

`enum` — Type-Safe Constants

```
from enum import Enum

class TriageLevel(Enum):
    IMMEDIATE = 1
    EMERGENT = 2
    URGENT = 3
    LESS_URGENT = 4
    NON_URGENT = 5
```

`concurrent.futures` — Parallel Execution

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(process_patient, patient_ids))
```

`typing.Protocol` (3.8+) — Structural Typing

```
from typing import Protocol

class Predictor(Protocol):
    def predict(self, x) -> list[float]: ...

# Any class with a predict() method satisfies this
# — more flexible than ABC (duck typing with type safety)
```

QUICK-REFERENCE CHEAT SHEET

Need	Module	Key Function(s)
Flatten lists	<code>itertools</code>	<code>chain</code> , <code>chain.from_iterable</code>
Group data	<code>itertools</code>	<code>groupby</code> (must pre-sort!)
All combinations	<code>itertools</code>	<code>combinations</code> , <code>product</code>
Slice iterators	<code>itertools</code>	<code>islice</code>
Cache results	<code>functools</code>	<code>lru_cache</code>
Freeze arguments	<code>functools</code>	<code>partial</code>
Fold to single value	<code>functools</code>	<code>reduce</code>
Preserve decorator metadata	<code>functools</code>	<code>wraps</code>
Structured data	<code>dataclasses</code>	<code>@dataclass</code> , <code>field</code> , <code>asdict</code>
JSON string I/O	<code>json</code>	<code>dumps</code> / <code>loads</code>
JSON file I/O	<code>json</code>	<code>dump</code> / <code>load</code>
Type safety	<code>typing</code>	<code>Optional</code> , <code>Union</code> , <code>Callable</code>
Pattern search (anywhere)	<code>re</code>	<code>search</code>
Pattern search (start only)	<code>re</code>	<code>match</code>
All pattern matches	<code>re</code>	<code>findall</code>
Date arithmetic	<code>datetime</code>	<code>timedelta</code>
Parse date string	<code>datetime</code>	<code>strptime</code> (<code>p</code> = parse)
Format date to string	<code>datetime</code>	<code>strftime</code> (<code>f</code> = format)
Production logging	<code>logging</code>	<code>getLogger</code> , <code>DEBUG</code> -> <code>CRITICAL</code>
Modern file paths	<code>pathlib</code>	<code>Path</code> , <code>/</code> operator, <code>glob</code>
Environment variables	<code>os</code>	<code>os.environ.get()</code>
Stream CSV rows	<code>csv</code>	<code>DictReader</code>
Custom context manager	<code>contextlib</code>	<code>@contextmanager</code>
Interface enforcement	<code>abc</code>	<code>ABC</code> , <code>@abstractmethod</code>
Count occurrences	<code>collections</code>	<code>Counter</code> , <code>most_common()</code>
Type-safe constants	<code>enum</code>	<code>Enum</code>
Parallel execution	<code>concurrent.futures</code>	<code>ThreadPoolExecutor</code>

TOP INTERVIEW GOTCHAS — Rapid Review

Gotcha	Why It Matters
--------	----------------

groupby requires pre-sorted data	Sort by key before calling groupby or you get fragmented groups
lru_cache needs hashable args	No lists or dicts as arguments — use tuples instead
Mutable default in dataclass	items: list = [] is a TypeError. Use field(default_factory=list)
dumps vs dump (json)	dumps/loads = strings. dump/load = files. Extra 's' = string
search vs match (re)	match only checks the beginning. search scans the whole string
strptime vs strftime	p = parse (string->datetime). f = format (datetime->string)
Type hints not enforced	typing is for tooling/docs only — no runtime enforcement
@wraps in decorators	Without it, function loses its __name__ and __doc__